Frontend Auth — Option C+D (NextAuth Credentials + ORCID)

Status: proposed (Architecture) **Scope:** Frontend authentication wiring for `malagasyfishes.org` against the existing Django auth surface on `api.malagasyfishes.org`. Two-gate split (MVP credentials → ORCID). **Supersedes:** the BFF-lite recommendation in `docs/planning/architecture/gate-08-auth-and-fetch.md` §1. That doc proposed a thin Next.js route-handler proxy. The owner has since chosen NextAuth + ORCID over rolling our own; this spec is the authoritative plan for that path. **Non-scope:** MFA, magic-link, institutional SSO/SAML, COORDINATOR_API_TOKEN deprecation, email-deliverability vendor selection.

---

1. The actual gap

The Django side is **done**. `backend/accounts/views.py` already ships `register/`, `verify/`, `login/`, `logout/`, `me/` with rate limiting, `TimestampSigner` email verification, generic-error responses for enumeration resistance, and DRF `Token` issuance. Tier gating (`backend/accounts/permissions.py`) is hardened — `TierPermission` and `TierOrServiceTokenPermission` both enforce `is_active` after PR #113. The `User` model has every field we need: `email` (unique), `name`, `password`, `access_tier` (1–5, default 2), `institution` FK, `expertise_areas`, **`orcid_id` already present**, `is_active` (default False), `is_staff`, `date_joined`.

The frontend has **none of it**. No `next-auth` dep in `frontend/package.json`. No `/login`, `/signup`, `/verify`, or `/account` route. `frontend/lib/api.ts` 's `apiFetch` is auth-blind — it does not forward an `Authorization` header, which is why the map page's tier-gate path (`SpeciesLocality` Tier 3 detail) is currently unreachable from the browser.

Read the rest of this spec as **a frontend integration project with two small Django additions** (an ORCID-link endpoint and one settings/CSRF tweak), not a greenfield auth build.

---

2. Library choice

`next-auth@4.24.x`, pinned to the latest 4.24 patch at install time.

- v4 is stable, ships first-class App Router support since 4.23, has years of production use, and aligns with our Next.js `14.2.15`.
- v5 / Auth.js is still in beta as of this writing and rebrands the import surface (`@auth/core`). We have a hard ABQ deadline (June 1–5, 2026) and zero appetite for chasing beta breakage.
- Pin (not caret) because auth library upgrades deserve a deliberate PR.

Add to `frontend/package.json` `dependencies`. No other runtime deps required (NextAuth ships its own JWT/JWE).

---

3. NextAuth configuration shape

File: `frontend/app/api/auth/[...nextauth]/route.ts` plus an `frontend/lib/auth.ts` exporting the `authOptions` object (so server components can `getServerSession(authOptions)`).

Providers

- **Credentials** — `authorize({email, password})` POSTs to `${process.env.NEXT_PUBLIC_API_URL}/api/v1/auth/login/` . On 200, parses `{token, access_tier, user_id}` and returns `{ id: String(user_id), email, tier: access_tier, drfToken: token }` . On 401/429, returns `null` so NextAuth surfaces the generic credentials error — preserves the enumeration-resistance Django already enforces. Do **not** leak the 429 as a distinct UI message; show the same "Invalid email or password" copy and let the rate-limit window pass.
- **ORCID** (Gate 12) — manual OAuth 2.0 provider config. Endpoint base `https://orcid.org` (prod) / `https://sandbox.orcid.org` (staging), scope `/authenticate` , callback `${NEXTAUTH_URL}/api/auth/callback/orcid` . After the OAuth dance, hand off to Django (see §6) and store the resulting DRF token on the JWT exactly like the Credentials path.

Session strategy

- `session: { strategy: "jwt" }` (default). We are **not** using NextAuth's database adapter — Django remains the user store.
- `jwt({ token, user, trigger })` callback persists `tier` , `drfToken` , and `tierFetchedAt` (epoch seconds) onto the JWT.
- `session({ session, token })` callback projects `tier` and `drfToken` onto the `Session` object so server components can read them.
- `pages: { signIn: "/login" }` so unauthenticated server-side `getServerSession` redirects land somewhere we control.

Cookies (prod)

```
__Secure-next-auth.session-token
  HttpOnly; Secure; SameSite=Lax; Path=/;
  Domain=.malagasyfishes.org
```

`Domain=.malagasyfishes.org` is the lever that lets the api subdomain read the cookie for SSR forwarding. In dev, NextAuth's defaults (`Domain` unset, no `Secure` , name `next-auth.session-token`) are correct — do not override.

---

4. Tier source of truth

The DRF `access_tier` is on the JWT for routine UX (badge rendering, route guards) — every request paying a `/me/` round-trip is unacceptable on Madagascar 3G.

Refresh rule in the `jwt` callback:

```
if (!token.tierFetchedAt || now - token.tierFetchedAt > 300) {
  // re-fetch /api/v1/auth/me/ with token.drfToken
  // overwrite token.tier; set token.tierFetchedAt = now
  // on 401 (token revoked / user deactivated): clear the JWT → forces re-login
}
```

Five-minute TTL means a tier promotion or `is_active=False` deprovision propagates within five minutes of the user's next request. Faster than that is a premium nobody is paying for; slower than that and a freshly-deactivated coordinator keeps Tier 3 reads. Pair with the existing audit-log signal so ops can see when a downgrade happened.

A 401 from `/me/` is the canonical "token died on the server" signal — clear the session, do **not** silently retry.

---

5. SSR fetch forwarding

`frontend/lib/api.ts:apiFetch` gains an optional `authToken` argument and an `authHeaders` accessor:

- Server components call `getServerSession(authOptions)` and pass `session?.drfToken` into `apiFetch(path, { authToken: session.drfToken })`.
- `apiFetch` adds `Authorization: Token ${authToken}` only when present. Anonymous (Tier 1) calls remain unauthenticated and unchanged.
- The map page's Tier 3 `SpeciesLocality` detail fetch (`frontend/lib/mapLocalities.ts`) becomes reachable once an authenticated session exists.
- The coordinator dashboard SSR (`frontend/lib/coordinatorDashboard.ts`) keeps using `COORDINATOR_API_TOKEN` for now. A follow-up gate flips it to per-user `session.drfToken`; `TierOrServiceTokenPermission` already accepts both, so the swap is mechanical.

Client components must never see `drfToken`. Expose `tier` (and at most a boolean `authenticated`) via a thin `useSession()` wrapper or pass tier down from server components — never put the DRF token in the React tree.

---

6. Django changes

Stays as-is

- All five `accounts/` views/URLs (`backend/accounts/views.py`, `backend/accounts/urls.py`).
- `RegisterSerializer` / `LoginSerializer` / `UserProfileSerializer` (`backend/accounts/serializers.py`).
- `TierPermission` / `TierOrServiceTokenPermission` (`backend/accounts/permissions.py`) — including the `COORDINATOR_API_TOKEN` branch.
- DRF auth class config (`backend/config/settings/base.py:141`) — `TokenAuthentication` is already in the default list.

Gate 12 addition: `POST /api/v1/auth/orcid-link/`

Contract (request body, all from the verified OAuth callback on the Next.js side — server-to-server, **never** from the browser):

```
{
  "orcid_id": "0000-0001-2345-6789",
  "orcid_email": "user@example.org",          // optional; ORCID may not return one
  "orcid_name": "Public Display Name",        // optional
  "link_to_user_id": 42                        // optional; present iff "Connect
ORCID" while logged in
}
```

Behavior:

1. If `link_to_user_id` is set and the request is authenticated as that user, set `User.orcid_id = orcid_id` (reject if already taken by a different user → 409). Return `{token, access_tier, user_id}` mirroring `/auth/login/`.
2. Else look up `User.objects.filter(orcid_id=orcid_id).first()`. If found and `is_active`, mint a Token, return same shape.
3. Else if `orcid_email` is present, look up by email. If found, attach `orcid_id` to that user (409 on `is_active=False`, since email-verification still gates login). Return Token.
4. Else (new user, no email from ORCID) return `409 EMAIL_REQUIRED`. The frontend collects an email, calls `/auth/register/` with a generated unguessable random password, then re-calls `/auth/orcid-link/` with `orcid_email` set. From there it's the email-verification flow as normal — the user never types a password.

Auth on the endpoint itself: `IsAuthenticated` for the `link_to_user_id` branch; an unauthenticated **shared-secret** check (a new `ORCID_LINK_SHARED_SECRET` env var, sent as `Authorization: Bearer …` from Next.js) for the create/login branches. The browser never hits this endpoint directly.

`User.orcid_id` already exists on the model — no migration needed. Add a `unique=True` constraint in a small migration as part of Gate 12.

---

7. CSRF

NextAuth handles CSRF on its own callbacks (`/api/auth/*`) via the double-submit cookie pattern — leave defaults alone.

The Credentials provider's POST to Django `/api/v1/auth/login/` is a server-to-server call from the Next.js node, not a browser form. Django CSRF is enforced on `SessionAuthentication`-marked requests; since this POST has no Django session cookie and the view's auth class resolves to `TokenAuthentication`-or-anonymous, **no CSRF token is needed and none is sent**. The view stays as-is — no `@csrf_exempt`, no `authentication_classes` override. This is the same regime that the existing `register/` and `verify/` endpoints already operate under.

If a future change adds `SessionAuthentication`-driven flows from the browser, revisit. Today: nothing to do.

---

8. Account flows

Email signup (Gate 11)

1. `/signup` form → POST `/api/v1/auth/register/` from the Next.js server (not from the browser, so we get a deterministic origin and rate-limit IP).
2. Render "Check your email" interstitial. Django's `send_mail` ships a link of the form `${FRONTEND_BASE_URL}/verify?token=<TimestampSigner>`.
3. User clicks → `/verify?token=…` page POSTs `{token}` to `/api/v1/auth/verify/`. On 200, redirect to `/login?verified=1` with a flash banner.
4. Login → NextAuth Credentials flow → session set → redirect to `callbackUrl` (validated, see §13).

Default tier remains 2 (Researcher) per `User.access_tier` default — confirmed in Appendix A.

ORCID-only signup (Gate 12)

ORCID returns the iD reliably, name sometimes, **email only with explicit user consent** — assume it's missing.

1. "Sign in with ORCID" → OAuth → Next.js server hits `/auth/orcid-link/`.
2. If Django returns 409 `EMAIL_REQUIRED`, redirect to `/signup/orcid?orcid=<iD>` with the iD in a short-lived signed Next.js cookie (NextAuth's pre-auth state cookie is the right vehicle).
3. User submits email; backend mints a registered-but-inactive user, attaches `orcid_id`, sends verification email. From here, identical to the email-signup path.
4. After verify → next ORCID sign-in lands on the existing-user branch in `/auth/orcid-link/` and a session is established.

Account linking (Gate 12)

Logged-in user clicks "Connect ORCID" on `/account` → ORCID OAuth → Next.js server hits `/auth/orcid-link/` with `link_to_user_id = session.user.id`. On 200, refresh session (re-fetch `/me/`).

---

9. Vercel preview behavior

Cookies on `Domain=.malagasyfishes.org` cannot be read from `*.vercel.app`. Preview deploys will therefore stay **anonymous**; that is intentional and acceptable.

- Authenticated UAT happens on a stable `staging.malagasyfishes.org` (or similar) hostname under the same registered domain. Wire that as a Vercel custom domain alias on the staging branch. CI/preview deploys can still smoke-test the login form rendering, just not a logged-in session.
- Document this in `OPERATIONS.md` so a reviewer who hits a preview URL doesn't assume auth is broken.

---

10. Migration plan

Frontend additions

- `frontend/lib/auth.ts` — `authOptions` export.
- `frontend/app/api/auth/[...nextauth]/route.ts` — handler.
- `frontend/app/login/page.tsx`, `frontend/app/signup/page.tsx`, `frontend/app/verify/page.tsx`, `frontend/app/account/page.tsx`.
- `frontend/lib/api.ts` — add `authToken` argument to `apiFetch`.
- `frontend/middleware.ts` — guard `/account` and any future tier-gated routes via NextAuth's `withAuth` helper.

Backend additions

- Gate 11: none. (Possibly tighten `verify/` to also return `access_tier` for a smoother post-verify autologin — optional.)
- Gate 12: `/auth/orcid-link/` view + URL + tests; `unique=True` migration on `User.orcid_id`; ORCID OAuth client app registered with ORCID; `ORCID_LINK_SHARED_SECRET` env var.

Migration risk: tiny. Production has zero users today (this is pre-launch). The `User` model gains nothing in Gate 11 and gains only a uniqueness constraint in Gate 12.

Rollout safety. Hide the login/signup nav links behind `NEXT_PUBLIC_FEATURE_AUTH=true`. `/login` and `/signup` routes can still exist for direct-link UAT; only the nav surface is flag-gated. If something breaks in the week before ABQ, flip the flag, ship anonymous-only, demo the read surface.

---

11. Gate split

Gate 11 — Auth MVP (Credentials only). Target pre-ABQ, ~3 weeks.

- NextAuth v4 install + Credentials provider.
- `/login`, `/signup`, `/verify`, `/account` pages.
- Session JWT carries `tier` + `drfToken`; 5-minute `/me/` refresh.
- `apiFetch` forwards `Authorization: Token …` from SSR.
- The Tier 3 `SpeciesLocality` detail path on the map page becomes reachable for a logged-in coordinator.
- Logout → DRF token deletion (already wired).
- Feature flag in nav.

Gate 12 — ORCID + polish. Target post-ABQ stretch, ~1 week.

- ORCID provider in NextAuth.
- `/auth/orcid-link/` Django endpoint + tests + `User.orcid_id` uniqueness migration.
- "Sign in with ORCID" button on `/login`; "Connect ORCID" on `/account`.
- Post-OAuth email-collection flow when ORCID withholds email.

Out of scope, both gates. MFA. Magic-link. Institutional SSO / SAML. `COORDINATOR_API_TOKEN` deprecation.

---

12. Test strategy

Backend

- Existing `accounts/` tests stay (`backend/accounts/tests/…`). Do not refactor.
- Gate 12: unit tests for `/auth/orcid-link/` covering each branch in §6 — link-existing-user, login-existing-user-by-iD, login-by-email, EMAIL_REQUIRED, iD collision (409), email collision with `is_active=False` (409).

Frontend

- Vitest covers `authorize()` shape, the `jwt` / `session` callbacks (mocked `/me/` for refresh), and the `apiFetch` `authToken` branch. No real Django round-trips in unit tests.
- Playwright e2e in `frontend/e2e/` covers signup → email link → verify → login → access a Tier-3-gated page and assert the request carried `Authorization: Token …`.

Adversarial pass (test-writer agent)

- Rate-limit bypass attempts on `/login` (X-Forwarded-For spoofing — Django's `_get_client_ip` already honors XFF; document the trust boundary at the edge proxy).
- Expired `verify/` tokens (>48h).
- Session fixation (NextAuth handles, but assert).
- ORCID iD collision with a different user's email (Gate 12).
- Open-redirect via `callbackUrl` and via the email `verification_url` query param (see §13).
- Deactivated user with a still-valid JWT — first `/me/` refresh inside the 5-min window must 401 and force re-login.

---

13. Security review checklist

- HttpOnly + Secure cookie flags in prod.

- `SameSite=Lax`, not `Strict` — coordinator SSR forwarding reads the cookie cross-origin from the api subdomain.
- DRF Token rotation on logout — already handled in `views.logout`.
- Rate limit shared across `/login` (existing 5/15min/IP); `/register` and `/verify` should reuse the same `_check_and_record_rate_limit` keyed differently to prevent cheap signup-flood.
- Account enumeration: register, login, verify all return identical generic responses on failure — already verified in `views.py`.
- Session fixation: NextAuth rotates the session token on sign-in.
- Open redirect: validate `callbackUrl` against `[FRONTEND_BASE_URL, "/"]`-prefix allow-list before redirect; same allow-list applied to any post-verify redirect target.
- ORCID `state` parameter: NextAuth generates and validates on the OAuth provider config — do not override.
- The `/auth/orcid-link/` shared secret never reaches the browser; it's a Next.js-server-only env var.

---

14. Operational

New env vars

- `NEXTAUTH_SECRET` — 32+ bytes, generated per environment. Required.
- `NEXTAUTH_URL` — `https://malagasyfishes.org` in prod, `http://localhost:3000` in dev.
- `ORCID_CLIENT_ID`, `ORCID_CLIENT_SECRET` — Gate 12. Sandbox credentials in dev/staging, prod credentials in prod.
- `ORCID_LINK_SHARED_SECRET` — Django side, Gate 12. Mirrored on Next.js as a server-only env var.

Add a section to `OPERATIONS.md` listing these alongside the existing `COORDINATOR_API_TOKEN`, `NEXT_REVALIDATE_SECRET`, etc., with rotation cadence (annually for `NEXTAUTH_SECRET`, on personnel change for shared secrets).

Cookie domain

- Prod: `.malagasyfishes.org` so `api.malagasyfishes.org` SSR can read it.
- Dev: leave NextAuth defaults (`localhost`, no domain).

Deploy order

For Gate 11 the order is irrelevant (no backend changes). For Gate 12: deploy Django `/auth/orcid-link/` **first**, then frontend NextAuth ORCID provider — because the frontend signup-with-ORCID path POSTs to that endpoint.

Appendix A — Open questions

- **ORCID email reliability.** Does ORCID return email reliably enough that we can skip the post-OAuth prompt? Likely no — email release is gated on user consent at the ORCID consent screen, and many users decline. Spec assumes "no" and ships the prompt. Revisit if Gate 12 telemetry shows ≥95% email coverage.
- **Default signup tier.** Decision: **2 (Researcher)**, matching `User.access_tier` default. Public profile reads do not require an account, so Tier 1 self-signup would be a no-op surface — no point. Tier 3+ is granted manually by an admin (Gate 11 ships no self-promotion path; this is intentional).
- **Email deliverability vendor.** Mailgun vs Resend vs SendGrid is out of scope here but blocks Gate 11 sign-off — flagged for OPERATIONS work. Today `EMAIL_BACKEND` defaults to console, which is not

acceptable for the ABQ demo if any reviewer creates an account.

Appendix B — Files touched at a glance

Frontend (Gate 11):

- `frontend/package.json` — pin `next-auth@4.24.x`.
- `frontend/lib/auth.ts` — `authOptions`.
- `frontend/app/api/auth/[...nextauth]/route.ts` — handler.
- `frontend/app/login/page.tsx`, `frontend/app/signup/page.tsx`, `frontend/app/verify/page.tsx`, `frontend/app/account/page.tsx`.
- `frontend/lib/api.ts` — `authToken` argument.
- `frontend/middleware.ts` — route guard.

Backend (Gate 12 only):

- `backend/accounts/views.py` — add `orcid_link`.
- `backend/accounts/urls.py` — wire `orcid-link/`.
- `backend/accounts/serializers.py` — `OrcidLinkSerializer`.
- `backend/accounts/migrations/00XX_orcid_id_unique.py` — uniqueness constraint.
- `backend/config/settings/base.py` — `ORCID_LINK_SHARED_SECRET`.

Operational:

- `OPERATIONS.md` — env-var section.